Assignment – 1

Name	Mohankumar Muthusamy
Student Number	A00049023
Email	mohankumar.muthusamy2@mail.dcu.ie
Course Code	CSC1141
Course Name	Concurrent Programming
Lecturer	David Sinclair
Date	16th March 2026
Code in Github	https://github.com/akpmohan07/my-masters/tree/main/concurrent_programming/ParallelPrimeFinder

1. Declaration of Academic Integrity:

I hereby declare that the submitted work, including the Java program and this Word document, are solely my own work, except for elements that are clearly identified, cited and attributed to other sources below.

I acknowledge that I have read and understood Dublin City University's policy on academic integrity.

Cited Sources / References:

Prime counting reference - **primes.utm.edu/howmany.html**
https://www.baeldung.com/java-executor-service-tutorial
https://www.khanacademy.org/computing/computer-science/cryptography/comp-number-theory/a/trial-division

2. Algorithm Description:

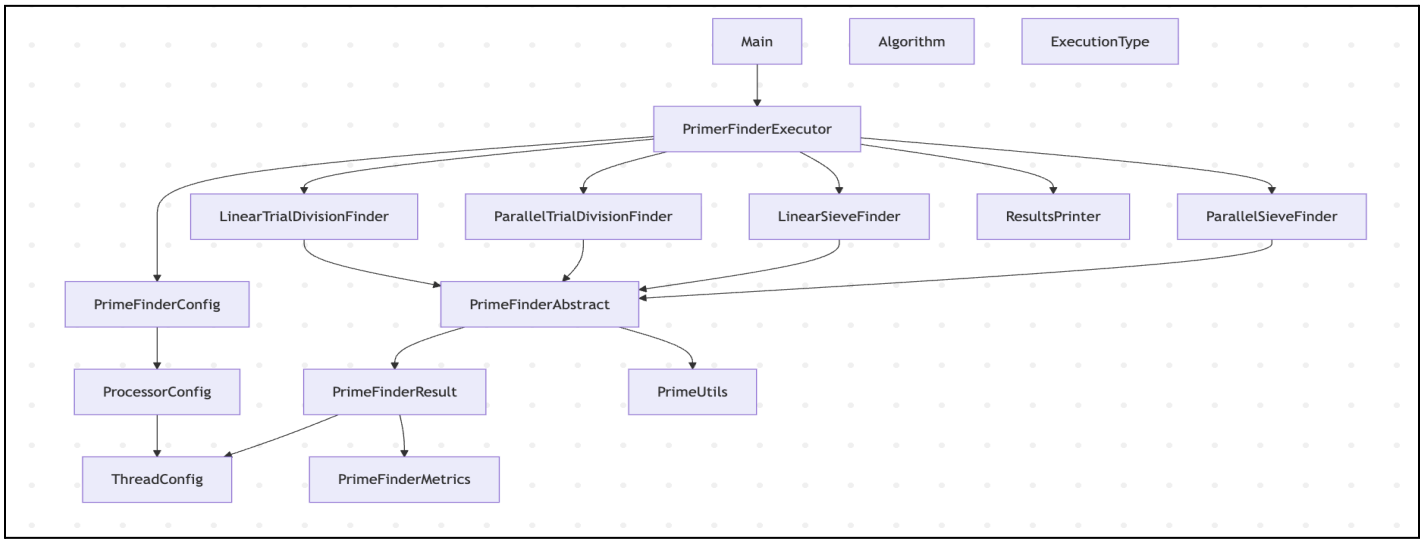
2.1. Overview:

This Java program finds all the prime numbers up to the user-defined N, which is a large number. Two core algorithms were implemented: Trial Division and the Sieve of Eratosthenes, each in both sequential and parallel versions, giving four implementations in total.

This problem is best suited to showcase multi-threading implementations by distributing the workload across the CPUs. The parallel versions distribute the workload across multiple CPU cores using Java's ExecutorService framework.

2.2. Class Design

The program follows the Template Method design pattern. An abstract base class, **PrimeFinderAbstract**, defines the execution flow timing, metrics collection, and result creation, while subclasses implement only the core computation logic.



2.3. Sequential Trial Division

The Trial Division algorithm tests each candidate number n in the range [2, N] for primality by checking whether any integer divides it evenly.

Algorithm:

```
Java
Primality check:  isPrime(n):
1. If n < 2 → not prime (handles 0, 1, negatives)
```

```
2. If n = 2 → prime (only even prime)
3. If n % 2 = 0 → not prime (all other even numbers). Early even rejection
where
    50 % of the numbers were rejected before running the loop.
4. For odd divisors d from 3 to sqrt(N), step 2: (Only odd numbers are
iterated)
   If n % d = 0 → not prime
5. If no divisor found → prime
```

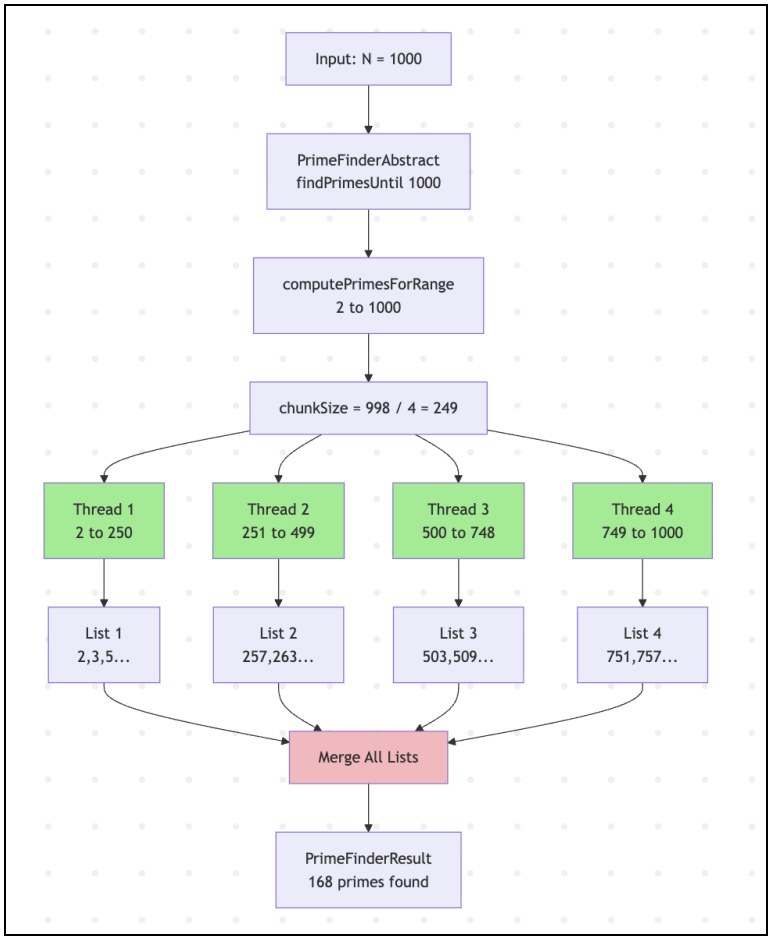
In this algorithm, each step is optimised for faster execution.

Time Complexity: O(N × sqrt(N))
Space Complexity: O(P) where P = number of primes found

2.4. Parallel Trial Division

Algorithm:

```Java
INPUT: N (upper limit), T (number of threads)

1. Split range [2, N] into T equal chunks
2. Create one task per chunk
3. Each task:
      for every number in its chunk:
           if isPrime(number) → add to local list
4. Run all tasks simultaneously
5. Wait for all tasks to finish
6. Merge all local lists into one
7. Return final prime list
```



Key design decisions:

1. Private lists per thread

Each thread have its own private list where it writes the result. And finally, all results are merged together. This eliminates the concurrent access issue of the result object.

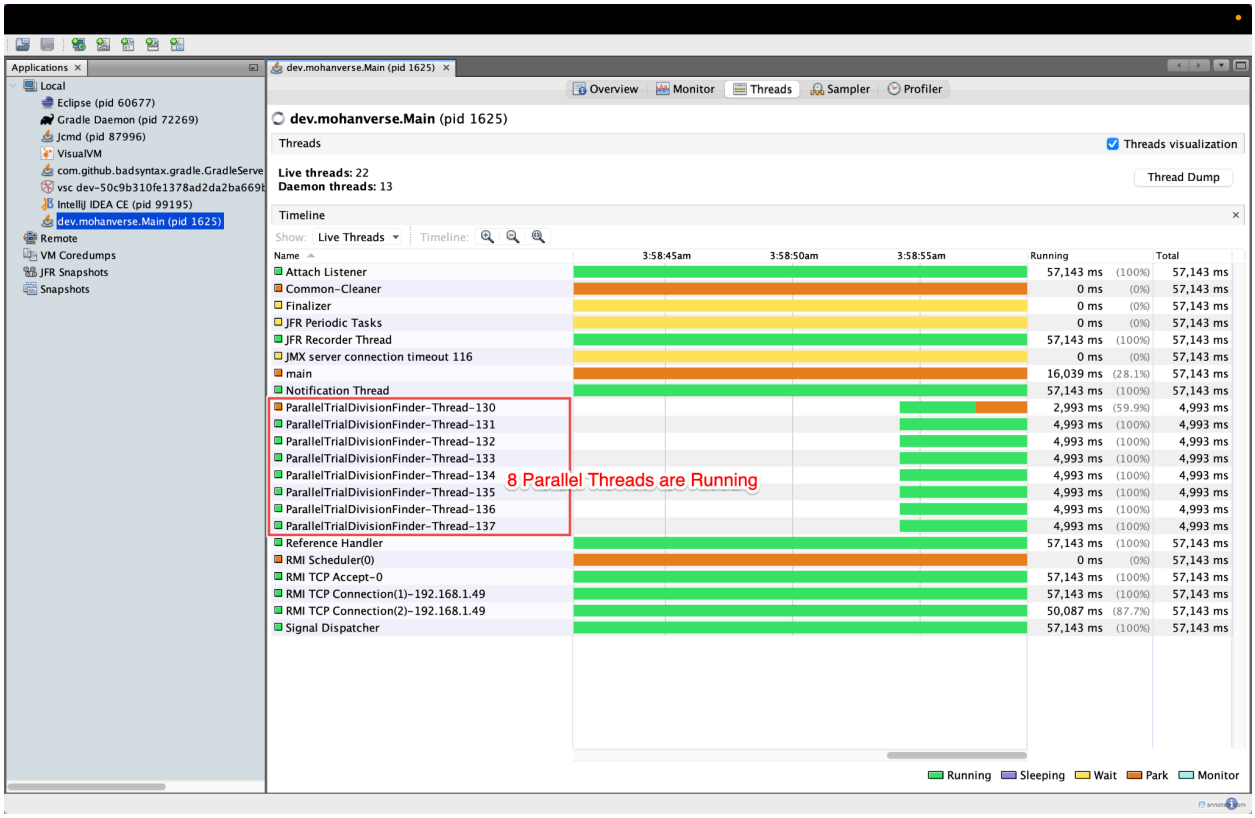
2. ExecutorService:

Provides a customised API for managing a thread pool, and invokeAll() for starting all the threads and waiting until all threads complete.

3. Range partitioning:

Based on the thread count availability, the list is divided into almost equal smaller chunks and assigned to the each thread via thread pool.

Time Complexity: O((N × sqrt(N)) / T) where T = thread count
Space Complexity: O(P), same as sequential



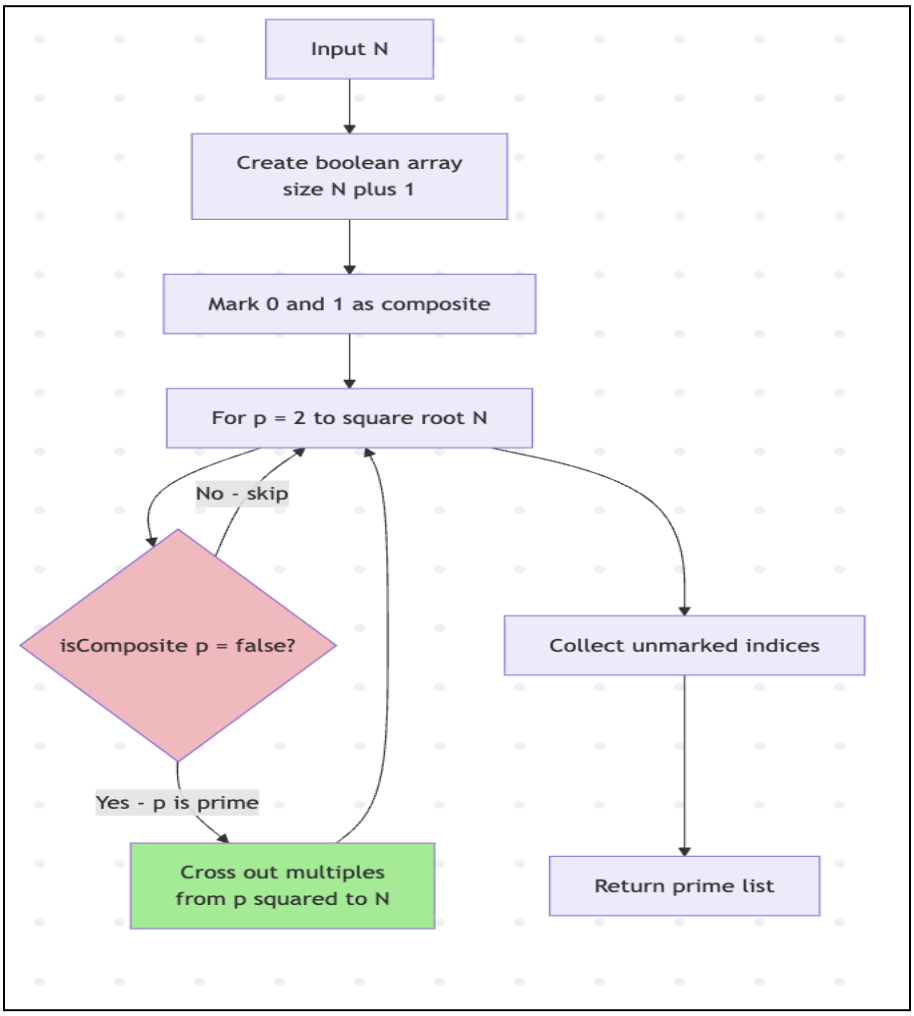
2.5. Sequential Sieve of Eratosthenes

The Sieve of Eratosthenes algorithm finds all primes up to N by marking composite numbers in a boolean array, rather than testing each number individually.

Algorithm:

```Java
INPUT: N (upper limit)

1. Create boolean array isComposite[0..N]
      all values set to false initially

2. Mark isComposite[0] = true
        isComposite[1] = true
      (0 and 1 are not prime)

3. For p = 2 to sqrt(N):
```

```
    if isComposite[p] = false:    (p is prime)
        for j = p² to N, step p:
            isComposite[j] = true

4. Collect all i where isComposite[i] = false
5. Return as prime list


Note: All smaller multiples already crossed by earlier primes
e.g. for p=5: 5×2, 5×3, 5×4 already marked by 2 and 3
First NEW multiple is always p×p
```

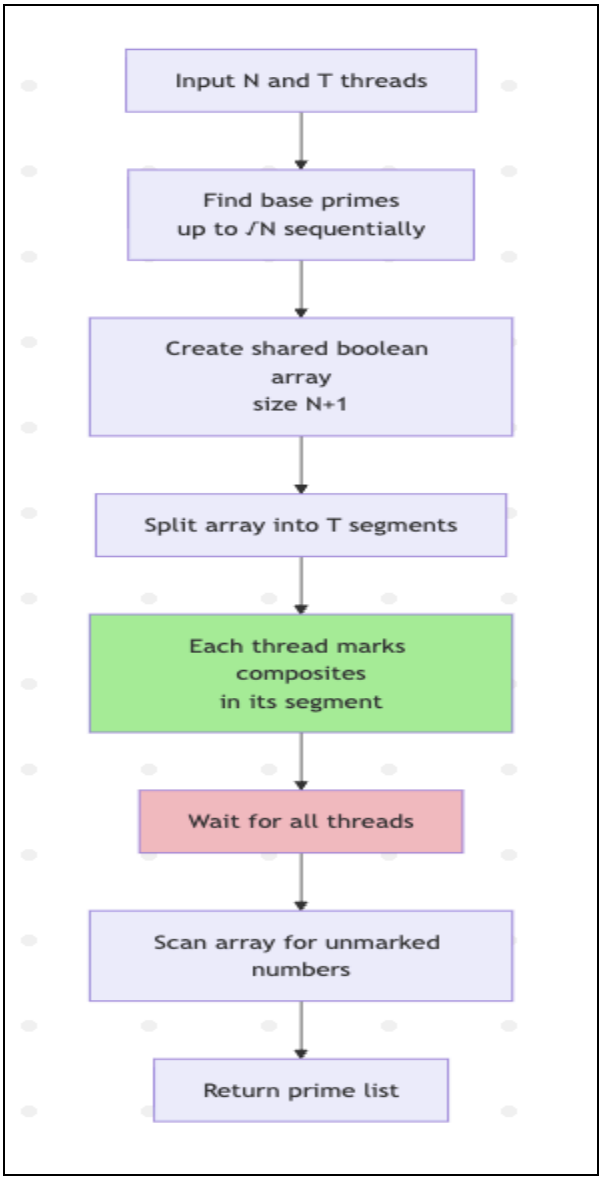


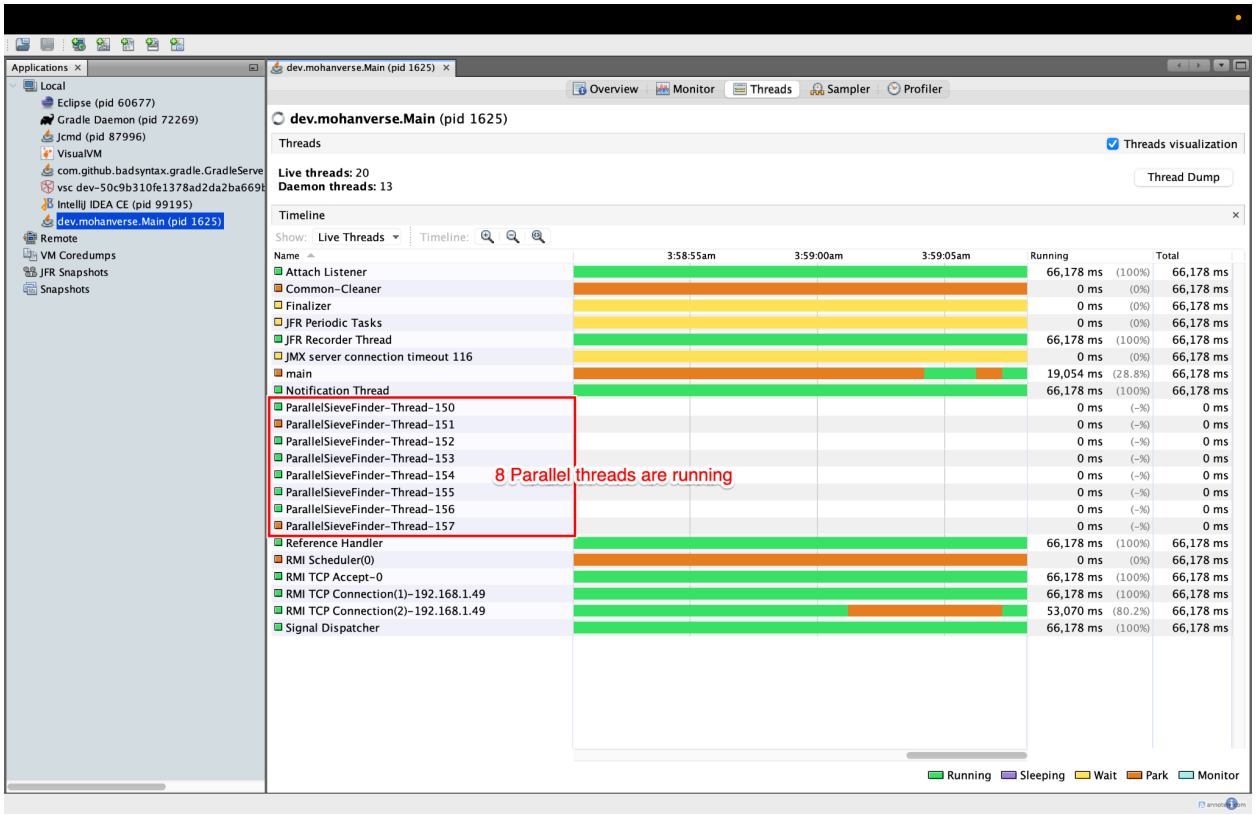
2.6. Parallel Sieve of Eratosthenes:

The parallel version shares a single boolean array across all threads and divides the marking work by range.

Algorithm

```
Java
INPUT: N, T threads

Step 1 – Find base primes up to sqrt(N)
         using LinearSieveFinder

Step 2 – Split array [2, N] into T segments

Step 3 – Each thread:
         for each base prime p:
             mark multiples of p in MY segment
                  as composite

Step 4 – Wait for all threads to finish

Step 5 – Scan full array
         collect unmarked numbers

Step 6 – Return prime list
```


Input N and T threads
Find base primes up to √N sequentially
Create shared boolean array size N+1
Split array into T segments
Each thread marks composites in its segment
Wait for all threads
Scan array for unmarked numbers
Return prime list



2.7. Algorithm Comparison:

Algorithm	Time	Space
Sequential Trial Division	O(N sqrt(N))	O(P)
Parallel Trial Division	O(Nsqrt(N) / T)	O(P)
Sequential Sieve	O(N log log N)	O(N)
Parallel Sieve	O(N log log N / T)	O(N)

Where: P = number of primes found, T = number of threads, N = upper limit

3. Testing & Results

3.1. Test Environment:

Hardware Overview:

Model Name:	MacBook Air
Model Identifier:	Mac14,2
Model Number:	MLXW3HN/A
Chip:	Apple M2
Total Number of Cores:	8 (4 performance and 4 efficiency)
Memory:	8 GB
System Firmware Version:	11881.140.96
OS Loader Version:	11881.140.96
Serial Number (system):	LG7CYV66HL
Hardware UUID:	3290D0E5-CFCF-5DE4-9635-125D0BA04040
Provisioning UDID:	00008112-0016248A0A6BA01E
Activation Lock Status:	Enabled

Software Overview:

Component	Details
Operating System	macOS, Sequoia, 15.6.1 (24G90)
JDK	Eclipse Temurin 21
Build Tool	Gradle 8.x (Kotlin DSL)
IDE	IntelliJ IDEA

Test Configurations:

Configuration	Values
N values tested	100,000 / 1,000,000 / 100,000,000
Thread percentages	25% / 50% / 75% / 100%
Thread counts	1/ 2 / 4 / 6 / 8
Algorithms tested	Trial Division / Sieve of Eratosthenes
Versions	Sequential + Parallel for each algorithm

3.2. Correctness Testing

Before performance testing, all four implementations were verified to produce identical prime counts for each value of N.

Results:

N	Sequential Trial Division	Parallel Trial Division	Sequential Sieve	Parallel Sieve	Match
100,000	9,592	9,592	9,592	9,592	✅
1,000,000	78,498	78,498	78,498	78,498	✅
100,000,000	5,761,455	5,761,455	5,761,455	5,761,455	✅

Verified against known mathematical values: https://primes.utm.edu/howmany.html

All versions produced identical results , correctness confirmed.

3.3. Performance Results:

N	Algorithm	Type	Threads	Core%	Time	Chunk Size	Primes
100,000	Linear Trial Division	Sequential	1	13%	8 ms	99,998	9,592
100,000	Sieve of Eratosthenes	Sequential	1	13%	4 ms	99,998	9,592
100,000	Linear Trial Division	Parallel	2	25%	11 ms	49,999	9,592
100,000	Linear Trial Division	Parallel	4	50%	3 ms	25,000	9,592
100,000	Linear Trial Division	Parallel	6	75%	2 ms	16,666	9,592
100,000	Linear Trial Division	Parallel	8	100%	2 ms	12,500	9,592
100,000	Sieve of Eratosthenes	Parallel	2	25%	8 ms	49,999	9,592
100,000	Sieve of Eratosthenes	Parallel	4	50%	5 ms	24,999	9,592
100,000	Sieve of Eratosthenes	Parallel	6	75%	7 ms	16,666	9,592
100,000	Sieve of Eratosthenes	Parallel	8	100%	4 ms	12,499	9,592
1,000,000	Linear Trial Division	Sequential	1	13%	48 ms	999,998	78,498
1,000,000	Sieve of Eratosthenes	Sequential	1	13%	5 ms	999,998	78,498
1,000,000	Linear Trial Division	Parallel	2	25%	25 ms	499,999	78,498
1,000,000	Linear Trial Division	Parallel	4	50%	16 ms	250,000	78,498
1,000,000	Linear Trial Division	Parallel	6	75%	15 ms	166,666	78,498
1,000,000	Linear Trial Division	Parallel	8	100%	19 ms	125,000	78,498
1,000,000	Sieve of Eratosthenes	Parallel	2	25%	8 ms	499,999	78,498
1,000,000	Sieve of Eratosthenes	Parallel	4	50%	7 ms	249,999	78,498
1,000,000	Sieve of Eratosthenes	Parallel	6	75%	7 ms	166,666	78,498
1,000,000	Sieve of Eratosthenes	Parallel	8	100%	7 ms	124,999	78,498
100,000,000	Linear Trial Division	Sequential	1	13%	25.93 sec	99,999,998	5,761,455
100,000,000	Sieve of Eratosthenes	Sequential	1	13%	874 ms	99,999,998	5,761,455
100,000,000	Linear Trial Division	Parallel	2	25%	15.64 sec	50,000,000	5,761,455
100,000,000	Linear Trial Division	Parallel	4	50%	11.09 sec	25,000,000	5,761,455
100,000,000	Linear Trial Division	Parallel	6	75%	9.77 sec	16,666,667	5,761,455
100,000,000	Linear Trial Division	Parallel	8	100%	8.82 sec	12,500,000	5,761,455
100,000,000	Sieve of Eratosthenes	Parallel	2	25%	1.49 sec	49,999,999	5,761,455
100,000,000	Sieve of Eratosthenes	Parallel	4	50%	1.51 sec	24,999,999	5,761,455
100,000,000	Sieve of Eratosthenes	Parallel	6	75%	1.71 sec	16,666,666	5,761,455
100,000,000	Sieve of Eratosthenes	Parallel	8	100%	1.70 sec	12,499,999	5,761,455
1,000,000,000	Linear Trial Division	Sequential	1	13%	13.12 min	999,999,998	50,847,534

3.4. Analysis and Findings:

1. Finding 1: Thread overhead at small N:

For N=100,000 and N=1,000,000, parallel versions were not much higher than the sequential, as the thread creation cost was included in the parallel versions.

2. Finding 2: Algorithm beats parallelisation:

The sequential sieve algorithm beats the 8-threaded linear trial division algorithm.

3. Finding 3: Parallel Sieve overhead:

In the sieve algorithm, the 8-core parallel mode(1.7sec) is slower than the sequential method(874ms), so it also has the thread cost. And to see the real difference of sieve , we need to run for a much bigger number.

4. Finding 4 — Apple M2 sweet spot at 4 threads:

The M2 Mac laptop has 4 Performance cores and 4 efficiency cores. We can see that when the thread count 4, it performs very well. Beyond that, performance reduces as the threads fight for the same physical cores.

4. Conclusions

This assignment implemented and compared four prime finding algorithms across sequential and parallel execution strategies.

Learnings:

1. Algorithm choice matters more than parallelisation.
2. Parallelisation has overhead costs for a smaller load.
3. Parallelisation provides meaningful speedup within the same algorithm for higher loads.
4. Hardware architecture significantly impacts results.
5. Memory is a real constraint for the Sieve algorithm.